FP-Style Parallel Quicksort

Because this uses a functional style already, it's now easy to convert this to a parallel version using futures, as shown in the next listing. The set of operations is the same as before, except that some of them now run in parallel. This version uses an implementation of the Quicksort algorithm using futures and a functional style.

Listing 4.13. Parallel Quicksort using futures

```
template<typename T>
std::list<T> parallel_quick_sort(std::list<T> input)
{
    if(input.empty())
    {
        return input;
    }
    std::list<T> result;
    result.splice(result.begin(),input,input.begin());
    T const& pivot=*result.begin();
    auto divide_point=std::partition(input.begin(),input.end(),
            [&](T const& t){return t<pivot;});

    std::list<T> lower_part;
    lower_part.splice(lower_part.end(),input,input.begin(),
        divide_point);

    std::future<std::list<T> > new_lower(                    <-1
        std::async(&parallel_quick_sort<T>,std::move(lower_part)));

    auto new_higher(
        parallel_quick_sort(std::move(input)));    <-2

    result.splice(result.end(),new_higher);        <-3
    result.splice(result.begin(),new_lower.get());       <-4
    return result;
}
```

The big change here is that rather than sorting the lower portion on the current thread, you sort it on another thread using `std::async()` ❶. The upper portion of the list is sorted with direct recursion as before ❷. By recursively calling `parallel_quick_sort()`, you can take advantage of the available hardware concurrency. If `std::async()` starts a new thread every time, then if you recurse down three times, you'll have eight threads running; if you recurse down 10 times (for ~1000 elements), you'll have 1024 threads running if the hardware can handle it. If the library decides there are too many spawned tasks (perhaps because the number of tasks has exceeded the available hardware concurrency), it may switch to spawning the new tasks synchronously. They will run in the thread that calls `get()` rather than on a new thread, thus avoiding the overhead of passing the task to another thread when this won't help the performance. It's worth noting that it's perfectly conforming for an implementation of `std::async` to start a new thread for each task (even in the face of massive oversubscription) unless `std::launch::deferred` is explicitly specified or to run all tasks synchronously unless `std::launch::async` is explicitly specified. If you're relying on the library for automatic scaling, you're advised to check the documentation for your implementation to see what behavior it exhibits.